

Summary: Generative Adversarial Network for MNIST Digit Generation

GitHub repository: <https://github.com/jlvi179/Advanced-Deep-Learning>

What the Notebook Does

This notebook trains a fully connected Generative Adversarial Network (GAN) to generate handwritten MNIST digits. It loads the MNIST training set with `torchvision`, normalizes the images to the $[-1, 1]$ range, trains a generator and discriminator in alternating optimization steps, and visualizes both the training losses and generated digit samples across epochs.

Architecture Choices

The main design choices are the input normalization, the MLP-based generator and discriminator, and the adversarial training objective:

- **Data preprocessing:** MNIST images are converted to tensors, normalized with mean and standard deviation 0.5, and flattened from 28×28 pixels into 784-dimensional vectors.
- **Generator:** random noise vectors of dimension 128 are mapped through fully connected layers with hidden sizes 256, 512, and 1024. Batch normalization and ReLU activations stabilize the hidden layers, while a final `Tanh` activation matches the normalized image range.
- **Discriminator:** flattened images are classified by an MLP with hidden sizes 1024, 512, and 256. LeakyReLU activations and dropout help the discriminator learn robust real-vs-fake decisions without becoming too brittle.
- **Training objective:** both networks use binary cross entropy. The discriminator is trained on real images labeled as 1 and detached generated images labeled as 0, while the generator is trained to make the discriminator classify generated images as real.

Fine points to watch:

- **Output range:** the `Tanh` output only makes sense because the real images are normalized to $[-1, 1]$; changing the preprocessing would require changing the generator output activation or scaling.
- **Training balance:** GAN training depends on keeping the discriminator and generator at comparable strength. Dropout in the discriminator and the non-saturating generator loss help avoid early collapse.
- **Detached fake images:** generated images must be detached during the discriminator update so gradients do not update the generator in that step.